DAY 08 OF 10

Object Oriented Programming

Model real-world things with classes and objects

Today's agenda

1. Classes and objects
2. __init__ and self
3. Methods and attributes
4. Inheritance and polymorphism
5. Encapsulation

Python in 10 Days

Lakshmi Prasad Gondinigari

Why OOP?

DAY 08

- Bundle data (attributes) and behaviour (methods) together
- A class is a blueprint; an object is one thing built from it
- Example: class Student; objects asha, ravi, sam
- Used in every large Python project and in libraries like Pandas and Scikit-learn
- You have been using objects already: strings, lists, dicts

Your first class

- class Name: defines the blueprint
- __init__ runs when an object is created (the constructor)
- self is the object itself; attributes are set with self.x
- Methods are functions inside a class
- __str__ controls what print() shows

Python

```python
class Student:
    school = "ABC Institute"      # class attribute

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def grade(self):
        return "Pass" if self.marks >= 40 else "Fail"

    def __str__(self):
        return f"{self.name} ({self.marks})"

s1 = Student("Asha", 82)
print(s1)
print(s1.grade())
```

OUTPUT

```
Asha (82)
Pass
```

Instance vs class attributes

- Instance attributes belong to one object (self.name)
- Class attributes are shared by all objects (school)
- Each object has its own copy of instance data
- Change shared data through the class name

Python

```
a = Student("Asha", 82)
b = Student("Ravi", 35)
print(a.name, a.grade())
print(b.name, b.grade())
print(a.school == b.school)
Student.school = "XYZ College"
print(b.school)
```

OUTPUT

```
Asha Pass
Ravi Fail
True
XYZ College
```

Inheritance

- A child class reuses and extends a parent class
- class Child(Parent):
- super() calls the parent's method
- Overriding: redefine a method in the child

Python

```python
class Person:
    def __init__(self, name):
        self.name = name
    def intro(self):
        return f"I am {self.name}"

class Teacher(Person):
    def __init__(self, name, subject):
        super().__init__(name)
        self.subject = subject
    def intro(self):
        return f"{super().intro()} and I teach {self.subject}"

print(Teacher("Lakshmi", "Python").intro())
```

OUTPUT

```
I am Lakshmi and I teach Python
```

Polymorphism

DAY 08

- Different classes can have the same method name
- Code can work with any object that has the method
- You do not need to know the exact class
- Makes programs flexible and easy to extend

Python

```python
class Dog:
    def speak(self):
        return "Woof"

class Cat:
    def speak(self):
        return "Meow"

for pet in (Dog(), Cat()):
    print(pet.speak())
```

OUTPUT

```
Woof
Meow
```

Encapsulation: protect your data

DAY 08

- Prefix _ means 'internal, please do not touch'
- Validate inside methods so the object stays in a valid state
- @property exposes a value like an attribute but read-only
- Real example: a bank account must never go negative

Python

```python
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self._balance = balance
    def deposit(self, amt):
        if amt <= 0:
            raise ValueError("Deposit must be positive")
        self._balance += amt
    def withdraw(self, amt):
        if amt > self._balance:
            raise ValueError("Insufficient funds")
        self._balance -= amt
    @property
    def balance(self):
        return self._balance

acc = BankAccount("Asha", 1000)
acc.deposit(500)
acc.withdraw(300)
print(acc.balance)
```

OUTPUT

```
1200
```

Practice time

DAY 08

1. Create a Rectangle class with area() and perimeter()
2. Create a Book class and print it nicely with __str__
3. Create Employee and Manager (inherits Employee, adds a team list)
4. Extend BankAccount with a transaction history list
5. Create a Counter class with increment() and reset()

Challenge: a Library class that manages many Book objects (add, search, list)

Day 8 recap

DAY 08

- class = blueprint, object = instance
- __init__, self, methods, attributes
- Inheritance with super(); polymorphism through shared method names
- Encapsulation with _name and @property

Homework: finish practice. Next: NumPy and Pandas